

Final Report — BackTestBench

Course: [Sum26] Practicum Project

Project: BackTestBench: modular backtesting and sandbox platform for trading strategies

Track: Industrial

TA: Gerald Imbugwa

Customer: Alexey Potemkin

Date: 20.07.2026

0. Links at a glance

Resource	URL
Repository	https://github.com/BackTest-bench-team/BackTestBench
Frozen commit	https://github.com/BackTest-bench-team/BackTestBench/commit/09027526330bc9b94208f302f669623168826bef
Deploy (course VM)	http://10.93.27.6/ — live dashboard; API docs: http://10.93.27.6/docs
Setup & run (README)	README.md · DOCKER.md
Docs hub	docs/README.md
UI design (Figma)	BackTestBench Design prototype
API documentation	docs/api_description.md · Swagger UI on course VM
Kanban	https://github.com/orgs/BackTest-bench-team/projects/1
Issues	https://github.com/BackTest-bench-team/BackTestBench/issues
Weekly demo videos	Submitted on the course portal for weeks 2–6 (see weekly PDFs in §8)
Final demo video	https://disk.yandex.ru/i/s9o3hXz79l9eXw · slides: Final presentation.pdf
This report (PDF)	Final report.pdf
Primary chat	https://t.me/Muratich_29

1. Team & workflow

- **Team name:** BackTestBench

- **Primary channel:** https://t.me/Muratich_29

Member	Role
Marat Diarov	Team lead — reports, customer communication, frontend
Georgii Beliaev	Tech lead — requirements & architecture, DevOps/QA
Bogdan Zinchenko	Data engineering and historical data management
Samy Badavi	Simulation Engine development
Maxim Kharlamov	Analytics and ranking
Philip Oghenerukevwe Idiare	Strategy module
Nikolay Taran	Broker adapter and T-Bank integration

How we worked

- Weekly customer demos/reviews with Alexey Potemkin plus mid-week sync; async execution via Telegram and GitHub Issues/[PRs](#).
- Kanban on GitHub Projects; milestones aligned to course weeks.
- Module ownership mapped to roles above (confirmed through Weeks 5–6); cross-cutting work (Docker, CI, docs, reports) owned by tech lead with frontend lead on dashboard UX.
- Tools: GitHub (repo, Issues, [PRs](#), Actions), Telegram, Docker Compose, pytest, Next.js toolchain.

1.1 Individual contributions (course summary, weeks 2–6)

Each row lists **top deliverables first** (what graders should open); the full PR audit trail for every member is in **§10.4**. Deliverables are cumulative across the practicum; authorship follows merge-commit branch history on the submission freeze (0902752).

Member	Role focus	Top 3 deliverables (click these)
Marat Diarov	Team lead · frontend · reports	1. Multi-strategy dashboard + bootstrap UX (#90 , #101–#102 , #151). 2. PM artifacts: Kanban , weekly PDFs (§8), this Final Report. 3. Multi-API tokens panel (#141) · Figma .
Georgii Beliaev	Tech lead · architecture · DevOps/QA	1. OpenAPI + Swagger UI (#150 , live). 2. GitHub Actions three-job CI (#137). 3. Architecture/docs hub (#75 , #138); Final Report freeze verification (§11).
Bogdan Zinchenko	Data engineering · historical data	1. Data Loader foundation (#97 : validation, SQLite cache). 2. Candle upsert reused by bootstrap/optimizer (§3.1). 3. Chunked fetch path integrated at freeze (#151).
Samy Badavi	Simulation engine	1. Core candle-by-candle engine (#82). 2. Optimizer execution engine (#128). 3. Trading-bot validation library (#144 ; backend only at freeze).

Member	Role focus	Top 3 deliverables (click these)
Maxim Kharlamov	Analytics · ranking	1. Top-N ranking + validation metrics (#100). 2. Optimizer ranked[] schema (#139). 3. Strategy health verdict on dashboard cards (#151).
Philip Oghenerukevwe Idiare	Strategy module	1. Composable YAML engine (#127). 2. Plugin strategies + parameter specs (#98). 3. Risk controls: constraints, trailing stop, filters (#142).
Nikolay Taran	Broker adapter · integrations	1. T-Bank historical candles (#88). 2. TwelveData / Bybit expansion (#135). 3. Binance adapter (#145); tokens panel \$10.1.

1.2 Deployment & reproducibility

Course VM (always-on demo)

- Dashboard: <http://10.93.27.6/>
- Swagger UI: <http://10.93.27.6/docs>
- Redeploy script (run on the VM): [reports/deploy-vm.sh](#)

One-command local rebuild

```
git checkout 09027526330bc9b94208f302f669623168826bef
cp .env.example .env    # set TINKOFF_TOKEN at minimum
docker compose up --build
```

Open <http://localhost> (or host port from APP_PORT, default 80). Stop: `docker compose down``.

Docker packaging (per component)

File	Purpose
Dockerfile.fullstack	Integrated Python 3.12 + Node 20 dashboard (app service)
Dockerfile	Backend-only image (backtest-app, test, dev profiles)
frontend/Dockerfile	Frontend production build (standalone server in full-stack image)
docker-compose.yml	Whole-project orchestration

Full service matrix, CI smoke steps, and VM hardening notes: [DOCKER.md](#). Quick start: [READEME.md](#).

1.3 Master branch stability

The submission freeze (0902752, 20.07.2026) is the tip of main at hand-in. That tree is **runnable** (docker compose up --build), **tested** (206 pytest passed, 78% src/ coverage – \$10.2), and **CI-verified** on every [PR](#) (backend-tests, frontend-checks, docker-smoke – \$10.1 CI screenshot). Weekly evolution on main is traceable through §8 and the linked weekly PDFs; post-W6 dashboard changes are summarized in §8 footnote.

2. Product overview & problem statement

Retail traders often evaluate strategies with fragmented broker terminals, ad-hoc scripts, and charting tools that do not form a reproducible research loop. Russian-market users can access data and trading through brokerage platforms (e.g. T-Investments) or build custom stacks, but lack a single modular environment for systematic backtesting, comparison, and validation.

BackTestBench is an industrial-track research platform that addresses that gap: deterministic, long-only backtests on historical OHLCV candles; multi-broker data adapters; a composable YAML strategy engine with optimization and ranking; strategy health verdicts; optional Live refresh on one strategy; and an integrated Next.js dashboard – packaged for reproducible local runs via Docker Compose.

3. Full product functionality

Capabilities below describe **current HEAD** (submission freeze 0902752), not only the shortest demo path. **Source of truth:** application code/tests plus the status matrix in [docs/README.md](#) (audited 20.07.2026). §3.2 copies that table so the report stays aligned with living docs.

3.1 Implemented (summary)

Market data & brokers

- Async broker adapters: T-Bank, TwelveData, Bybit, Binance (historical candles).
- Dashboard multi-API token UI ; env vars TINKOFF_TOKEN, TWELVEDATA_TOKEN, BYBIT_TOKEN, BINANCE_TOKEN.
- Data Loader: candle validation, SQLite upsert, chunked fetch and cache reuse for bootstrap and optimizer.

Strategies

- Composable YAML engine: series, rules, actions, TP/SL, constraints, time filters, trailing stop, drawdown guard, trend filter.
- Legacy plugin strategies still in tree: ma_crossover, ma_rsi, rsi_threshold (registry / auto-discovery).
- Parameter optimizer: grid and random sample.

Execution & analytics

- Candle-by-candle long-only simulation; commission and slippage; force-close on final candle.
- TradeLog, equity curve, portfolio state; metrics: total P&L, Sharpe, max drawdown, win rate, deposit baseline, return %.

- Strategy Top-N ranking and optimizer parameter ranking .
- Strategy health verdict (PASS / CAUTION / FAIL).

Dashboard & ops

- Next.js dashboard: bootstrap Run/Stop with progress bar, strategy cards, optimization panel, ranking badges, charts with buy/sell markers and buy-and-hold benchmark, optional **Live refresh** (one strategy). UX reference: [Figma design prototype](#).
- Configurable context via config/dashboard.json (instrument, timeframe, lookback, optimization mode).
- **API documentation:** implemented Next.js App Router routes (/api/dashboard, /api/bootstrap, /api/run-progress, /api/live-run, /api/tokens, ...) in [docs/api_description.md](#). Interactive OpenAPI: <http://10.93.27.6/docs>. Target FastAPI service in src/api remains **Planned**.
- Docker Compose integrated stack; GitHub Actions on ubuntu-latest: backend-tests, frontend-checks, docker-smoke ([PR #137](#)).

3.2 Implemented vs planned

Same matrix as [docs/README.md – Implemented vs planned](#):

Area	Status
Composable YAML strategies (config/strategies/*.yaml)	Implemented
Trigger/action rules with TP/SL and priority ordering	Implemented
Constraints, time filters, trailing stop, drawdown/trend guards (#142)	Implemented
Parameter optimizer (grid + random sample)	Implemented
Optimizer parameter ranking (ranked[], PR #139)	Implemented
Strategy Top-N ranking (build_top_n)	Implemented
Dashboard with Run/Stop, optimization panel, Live refresh (one strategy)	Implemented
Bootstrap progress bar (GET /api/run-progress, chunked fetch + backtest phases)	Implemented
Strategy health verdict (PASS / CAUTION / FAIL)	Implemented
Per-strategy enabled flag for next bootstrap (PATCH /api/strategies/{id})	Implemented
Chunked candle loading + SQLite cache (src/data_loader/backtest_fetch.py)	Implemented
Multi-API data sources (T-Bank, Twelve Data, Bybit, Binance) + token UI	Implemented
Instrument dropdown (single-instrument)	Partial
Commission / slippage in execution engine	Implemented

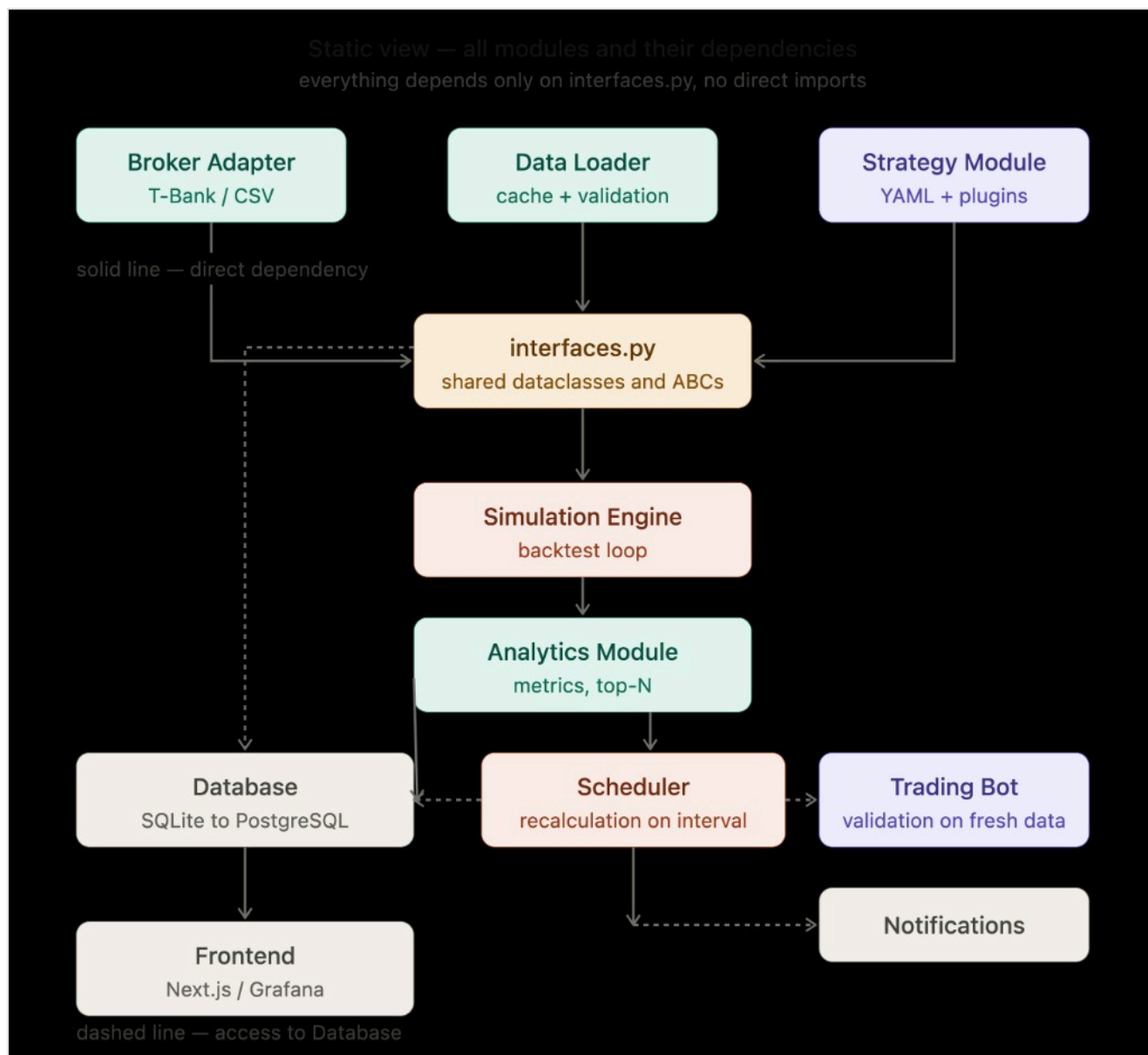
Area	Status
GitHub Actions CI (backend-tests, frontend-checks, docker-smoke on ubuntu-latest)	Implemented
Plugin strategies (ma_crossover, ma_rsi, rsi_threshold)	Implemented (codebase; dashboard uses composable YAML)
Explore dock + /api/explore	Removed
Trading bot dock + /api/bot	Removed
src/stability.py window explore analytics	Removed
Full multi-period / walk-forward stability ranking	Planned
End-to-end holdout validation workflow (second stage)	Not integrated
FastAPI service (src/api)	Planned
Relational run/trade/metrics persistence	Planned
Multi-instrument portfolio UI	Planned
CSV adapter, T-Bank place_order / get_portfolio, scheduler, notifications, live order automation	Planned / stubs

4. Architecture

Two views describe the **shipped product** at the submission freeze: a module map (§4.1) and the backtest execution flow (§4.2). Week 2 diagrams included a scheduled top-N refresh cycle, Explore/Bot workflow docks, and Grafana/PostgreSQL targets – those are **not** part of the delivered system at 0902752 (see §3.2 and §9).

4.1 Static view – modules and dependencies

Modules communicate through shared interfaces (interfaces.py / dataclasses), not direct cross-imports. Solid lines = dependency; dashed lines = database access.

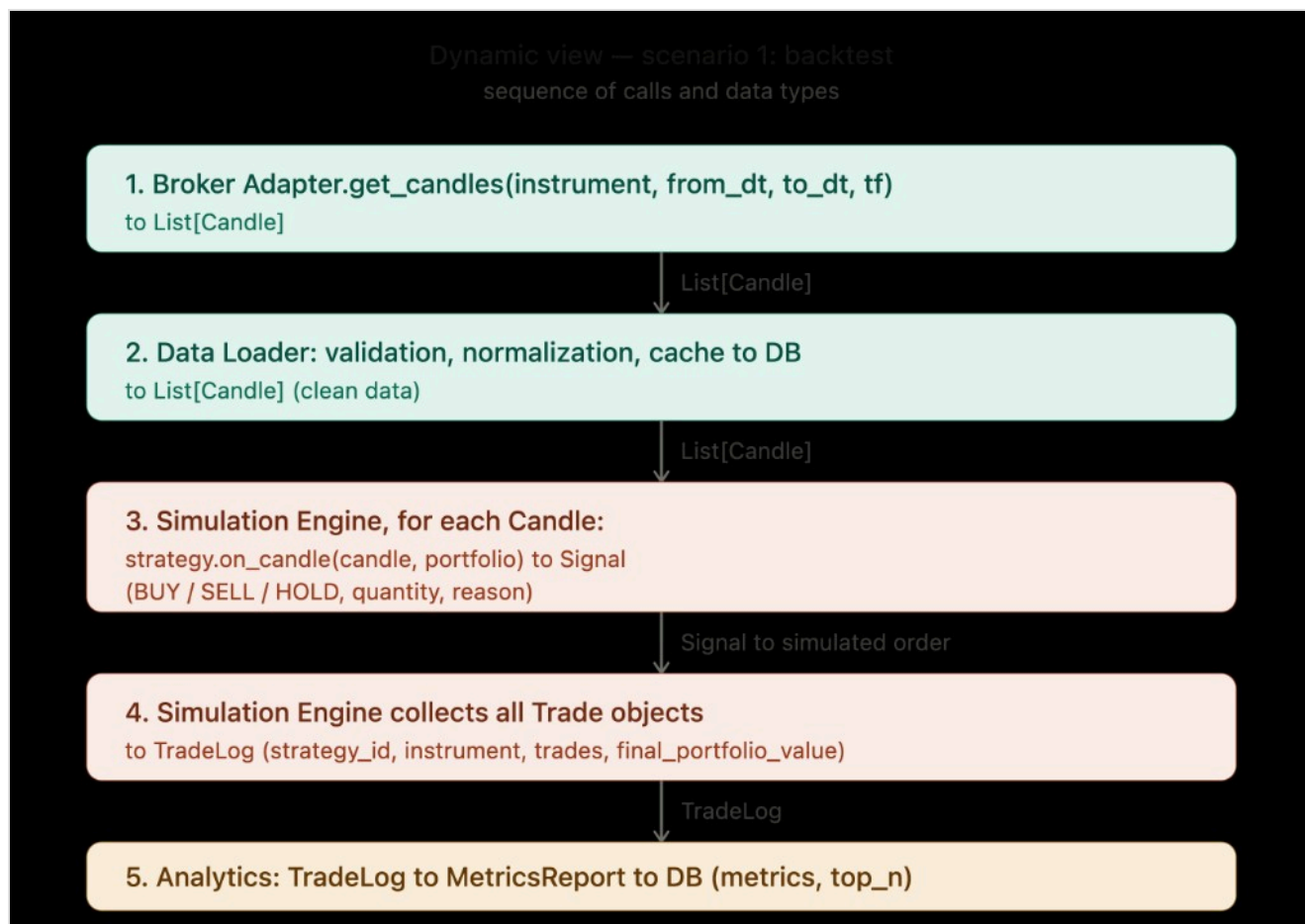


Static architecture — modules and dependencies

As shipped: Broker Adapter, Data Loader, Strategy (composable YAML + plugins), Simulation Engine, Analytics, and Frontend (Next.js dashboard) are live. Candle cache is **SQLite** (not PostgreSQL). Bootstrap backtests, Top-N ranking refresh, and optional Live refresh are triggered from the dashboard — there is **no** Scheduler timer, Explore/Bot dock, Notifications module, or Grafana in the running stack.

4.2 Dynamic view — backtest run

End-to-end path for a research backtest (optimizer and multi-strategy runs reuse the same engine and data-loader steps):



Dynamic architecture – backtest run

Runtime integration: the dashboard calls Next.js API routes (/api/dashboard, /api/bootstrap, /api/run-progress, /api/live-run, /api/tokens, /api/refresh-ranking, ...), which spawn main.py subcommands. Job progress and metrics are stored in JSON files under data/; candles are cached in SQLite. Full route list: [docs/api_description.md](https://10.93.27.6/docs/api_description.md). Interactive OpenAPI: <http://10.93.27.6/docs>.

4.3 Main components

Component	Responsibility
src/broker_adapter	Market-data adapters; factory by source
src/data_loader	Validation, normalization, chunked fetch, SQLite reuse
src/strategy + composable/	Strategy contracts, YAML compile, plugins
src/engine	Simulation, optimizer engines; trading-bot library (no UI)
src/analytics	Metrics, Top-N, optimizer ranking, health verdict
frontend/	Next.js UI + App Router API routes that spawn Python
main.py	Orchestration for bootstrap / ranking / live refresh
Docker / GH	Reproducible run and PR verification

4.4 Key decisions and trade-offs

Decision	Trade-off
Next.js routes as integration façade instead of shipping FastAPI first	Faster integrated UI delivery; JSON job files are ephemeral and not multi-user durable
Composable YAML strategies over only hardcoded plugins	Flexibility for customer exit/filter design; preset wiring bugs harder to audit
Long-only simulated execution, no live orders	Safe research sandbox; no production trading claims
SQLite candle cache + chunked boots trap fetch	Strong for optimizer reruns and Live ticks; not a full research warehouse
Simplified dashboard (removed Explore/Bot docks, July 2026)	Clearer MVP2 loop on one screen; multi-period / OOS validation stays roadmap (\$9)
GitHub-hosted CI (three jobs) after self-hosted smoke	More portable PR checks; relies on GHA quota/environment

Domain invariants retained: chronological candles, fresh strategy instance per run, deterministic strategy evaluation without I/O, engine owns portfolio mutation, forced close on last candle, P&L vs portfolio reconciliation within analytics tolerance.

5. Competitor / alternative analysis

Problem space: reproducible strategy research on market data with comparison/validation UX.

Alternative	Strengths	Gaps vs our problem
Backtrader / vectorbt / similar OSS engines	Mature Python simulation; rich community	DIY UI/ranking/validation; not M0EX/T-Bank-first; no course Docker MVP2 loop
QuantConnect / Lean	Cloud scale, multi-asset research	Heavy cloud/product lock-in; steeper ops; less fit for local industrial customer sandbox
TradingView Strategy Tester	Excellent charts UX, huge user base	Pine ecosystem; weak fit for local multi-broker adapter research and YAML optimizer ranking
Broker terminals (e.g. T-Investments) + sandbox	Native Russian market access and trading	Focused on trading/access, not modular strategy comparison platform
Custom Jupyter + pandas	Maximum flexibility	High engineering cost; weak shared UX, ranking, and reproducible team deploy

Alternative	Strengths	Gaps vs our problem
TSLab / professional desktop builders (brief)	Visual strategy construction	Commercial desktop focus; different audience than our Docker/GitHub MVP

Existing products give market access or programmable engines, but rarely a **local, modular, customer-driven Russian-market research bench** with composable strategies, optimizer ranking, strategy health verdicts, and a Dockerized dashboard in one Compose stack.

6. Differentiation

What stands out for this industrial project:

1. **Customer-aligned MOEX / multi-broker path** – T-Bank-first history, later TwelveData / Bybit / Binance through one factory and dashboard tokens.
2. **Composable YAML strategies + parameter optimizer ranking** – trigger/action rules with TP/SL, constraints, trailing/filters, and optimization.ranked[] for candidate comparison.
3. **End-to-end research loop in one UI** – bootstrap backtest → Top-N ranking → parameter search table → strategy health verdict; optional Live refresh on one strategy – not only a single hard-coded backtest script.
4. **Industrial evidence culture** – weekly customer reviews, measured pytest/CI baselines, honest Implemented/Planned/Removed boundaries (no fake live trading).
5. **Course-operable packaging** – Docker Compose + GitHub Actions three-job matrix + Open API/Swagger at <http://10.93.27.6/docs> for reproducible demos.

We do **not** claim to replace QuantConnect or TradingView on breadth; we claim a focused, reproducible sandbox tailored to the customer's local research workflow.

7. Research & validation (industrial)

7.1 Studies, experiments, feedback

- **Customer sessions** ([transcriptions/](#)): requirements and architecture (June), MVP demos (June–July), composable-engine / stability focus (13.07.2026).
- **Weekly status reports** (W1–W6, [reports/](#) PDFs in §8): milestone evidence, [PR](#) indexes, verification commands.
- **Technical experiments**: strategy plugin model → composable YAML; ranking library → UI Top-N → optimizer ranked[] → strategy health verdict; validation metrics library (backend only; Week 6 Explore/Bot UI removed on submission HEAD).
- **Quality gates**: pytest growth and CI evolution (self-hosted smoke → ubuntu-latest matrix).

7.2 Baseline measurements & impact

Base line	Early	Week 6 (b8408e9)	Submission freeze (0902752)	Impact
Backend tests	Week 3–4: ~66 passing (30.06)	188 / 77% (14.07)	206 / 78% (20.07)	Broader engine/composable/adapters/live-run coverage
CI	Self-hosted Docker smoke	GHA backend-tests + frontend-checks + docker-smoke	Same three-job matrix	Portable PR verification
Product surface	Single hardcoded MA Crossolver path (W3)	Composable YAML + optimizer + multi-API + explore/bot docks	Refactored dashboard: bootstrap progress, health, Live refresh; docks removed	Customer can search params, compare ranked cards, discuss risk with health verdicts – not OOS UI yet
Customer priorities	Extensibility & ranking (W4); composable engine (W5)	Stability proof & preset wiring (13.07)	Same P0 roadmap (§9)	Roadmap reframed to validation-quality, not more demo features

Documented product impact (qualitative): from a one-strategy MVP to a research bench the customer can use for parameter search and risk-aware design discussions. July 13 feedback explicitly reframed remaining work toward preset-runtime correctness and multi-period stability – not feature sprawl.

Customer-facing impact (industrial workflow):

- The customer now compares **three composable strategies on one dashboard** (parameter edit → Run backtest → ranked cards) instead of ad-hoc scripts per strategy – validated in the 29.06 and 13.07 review sessions ([transcriptions/](#)).
- **Parameter search + strategy health verdicts** (PASS / CAUTION / FAIL) give a shared vocabulary for risk discussions before any live-trading decision – the customer asked for stability proof, not more demo widgets (13.07).
- **Multi-API tokens panel** lets the customer switch T-Bank / TwelveData / Bybit / Binance in one UI for the same YAML strategies – reducing the “one broker, one notebook” fragmentation described at kickoff.

8. Evolution over the course (weeks 2–6)

Week 1 (foundation): team, customer kickoff, roles, repository/kanban, initial requirements (historical PDF retains early roster notes; this Final Report uses the active seven-person roster above).

Week	Focus	Shipped	Honest gap carried
2 (16.06)	Architecture & plan	Module contracts, schemas, week road map, industrial problem framing	No end-to-end run yet
3 (23.06)	MVP1 core	T-Bank → MA Crossover → engine → analytics → dashboard; Docker/CI start	One strategy, mostly hardcoded inputs
4 (30.06)	Extensibility & ranking	Plugins (MA/RSI), Top-N, multi-strategy dashboard, data-loader cache, configurable context	Validation = library only; flexible exits still missing
5 (07.07)	MVP2	Composable YAML engine, optimizer, Run/Stop UX, data-loader reuse, TwelveData/Bybit, GHA CI	Full validation workflow & walk-forward still open; T-Bank-only fetch initially for dashboard
6 (14.07)	Final touches	Optimizer ranked[], multi-API tokens, constraints/trailing, explore+stability, trading-bot jobs, Binance, frontend workflow docks	Full walk-forward ranking Planned; preset wiring Partial; order APIs still stubs

Post-Week 6 delta (19–20.07, submission freeze 0902752): dashboard refactor – bootstrap progress bar, chunked candle fetch, strategy health verdict, Live refresh ([PR #151](#)); OpenAPI spec + Swagger UI at <http://10.93.27.6/docs> ([PR #150](#)); Explore/Bot docks and src/stability.py **removed** from the UI (see §3.2). Week 6 explore/bot UI is historical evidence in [Week 6 report.pdf](#), not the submission UI.

Weekly PDFs remain dated submissions:

- [Week 2](#)
- [Week 3](#)
- [Week 4](#)
- [Week 5](#)
- [Week 6](#)

9. Future roadmap

If development continued beyond the course window:

Near-term (P0 from 13.07 customer review)

1. Audit and fix param_presets.yaml → runtime wiring so TP/SL and params always apply.
2. Multi-period / walk-forward stability checks for top optimized configs + stability column in ranking.
3. Percentage and risk/reward metrics alongside absolute P&L.

Medium-term

- Durable run history (relational storage) and/or FastAPI service behind the UI.
- Complete instrument picker UX; broaden multi-instrument research safely.
- Strengthen data-quality pipeline (duplicates / missing beyond current OHLC guards).
- Keep broker order/portfolio APIs stubbed until sandbox research hardening is done
 - **live order automation is far-future only.**

Explicitly out of near-term scope: production live trading, claiming profitability, replacing commercial cloud platforms.

10. Evidence appendix

10.1 Screenshots (embedded evidence)

All figures below are embedded in this report (not external links). **UI captures were shot on 20.07.2026** from a running docker compose stack at submission freeze 0902752. Architecture diagrams are in §4.1–§4.2. Figma reference: [prototype link](#).

Product UI (submission freeze)

TRADING ANALYTICS

BackTestBench

Instrument: LKOH

TF: 1m

Source: T-Bank

FIND STRATEGY

Name or ID, e.g. MA Crossover

Go to

Enter a name or ID and press Enter

BACKTEST CONTROL

Run settings

DATA SOURCE

T-BANK

READY

MOEX TQBR shares via Tinkoff Invest API

Paste new token to replace saved k

Save

Saved as t.7x...l93w

TWELVE DATA

TOKEN REQUIRED

Global equities, FX and crypto via Twelve Data API

Select to add API token

BYBIT

PUBLIC API

Crypto spot pairs via Bybit public kline API

No API key needed

BINANCE

PUBLIC API

Crypto spot pairs via Binance public kline API

No API key needed

INSTRUMENT

LKOH

Russian equities on MOEX (SBER, GAZP, ...)

TIMEFRAME

1m

Minimum resolution 1m. Lookback is limited only by broker history and disk space.

LOOKBACK DAYS

4

History window loaded for the backtest

INITIAL CAPITAL (RUB)

10000000

Starting portfolio cash for each strategy

COMMISSION %

0.05

Per-side fee as percent of notional (MOEX default 0.05%)

SLIPPAGE %

0.01

Price adjustment on each fill (default 0.01%)

OPTIMIZATION

GRID

Full grid search - evaluates every valid parameter combination from YAML choices. Guarantees the best result on the current dataset; can take several minutes.

SAMPLE

Random sample - evaluates N random combinations (see iterations + seed). Faster, but does not guarantee the global optimum.

TRIALS16

Ready to run

LKOH · 1m · 4d lookback · 16 trials

Run backtest

STRATEGIES

Registered from config/strategies/

Add strategy

Run

#1

RSI Bounce Demo

rsi_bounce_demo

GO LIVE

Delete

READY

+0,3 %

RSI PERIOD

20

RSI OVERSOLD

20

RSI OVERBOUGHT

60

STOP LOSS PCT

3

ORDER SIZE

1

PARAMETER SEARCH

Random sample: 16 of 108 combinations (seed 42)

#	P&L	RETURN %	SHARPE	PARAMS
1	34 380 P	+0,3 %	0,69	rsi period=20, rsi oversold=20, rsi overbought=60, stop loss pct=3
2	-348 051 P	-3,5 %	-4,29	rsi period=14, rsi oversold=20, rsi overbought=60, stop loss pct=3
3	-408 793 P	-4,1 %	-5,7	rsi period=20, rsi oversold=20, rsi overbought=65, stop loss pct=7
4	-437 763 P	-4,4 %	-5,12	rsi period=20, rsi oversold=25, rsi overbought=65, stop loss pct=5
5	-649 919 P	-6,5 %	-6,92	rsi period=9, rsi oversold=20, rsi overbought=65, stop loss pct=5

STRATEGY HEALTH

Return +0,3 % PF 1,15 Consistency 33,3 % vs B&H +8,7 % vs Deposit +0,2 %

Low period consistency

CAUTION

RETURN

+0,3 %

SHARPE

0,69

DRAWDOWN

2,3 %

WIN RATE

53,3 %

PROFIT FACTOR

1,15

CALMAR

12,21

CONSISTENCY

33,3 % (1/3 periods)

VS BUY&HOLD

+8,7 %

VS DEPOSIT

+0,2 %

Positive months

13% baseline

Main dashboard – bootstrap control panel, strategy cards, ranking, charts

TRADING ANALYTICS

BackTestBench

Instrument: LKOHTF: 1mSource: T-Bank

FIND STRATEGY

Name or ID, e.g. MA Crossover

Go to

Enter a name or ID and press Enter

BACKTEST CONTROL

Run settings

DATA SOURCE

T-BANK

READY

MOEX TQBR shares via Tinkoff Invest API

Paste new token to replace saved k

Save

Saved as t.7x...l93w

TWELVE DATA

TOKEN REQUIRED

Global equities, FX and crypto via Twelve Data API

Select to add API token

BYBIT

PUBLIC API

Crypto spot pairs via Bybit public kline API

No API key needed

BINANCE

PUBLIC API

Crypto spot pairs via Binance public kline API

No API key needed

INSTRUMENT

LKOH

Russian equities on MOEX (SBER, GAZP, ...)

TIMEFRAME

1m

Minimum resolution 1m. Lookback is limited only by broker history and disk space.

LOOKBACK DAYS

4

History window loaded for the backtest

INITIAL CAPITAL (RUB)

10000000

Starting portfolio cash for each strategy

COMMISSION %

0.05

Per-side fee as percent of notional (MOEX default 0.05%)

SLIPPAGE %

0.01

Price adjustment on each fill (default 0.01%)

OPTIMIZATION

GRID

Full grid search - evaluates every valid parameter combination from YAML choices. Guarantees the best result on the current dataset; can take several minutes.

SAMPLE

Random sample - evaluates N random combinations (see iterations + seed). Faster, but does not guarantee the global optimum.

TRIALS16

Ready to run

LKOH · 1m · 4d lookback · 16 trials

Run backtest

Multi-API data sources and instrument / timeframe row

Verification evidence

```

$ pytest tests -q
===== test session starts =====
platform darwin -- Python 3.12.2, pytest-8.3.4, pluggy-1.6.0
rootdir: /Users/georgijbelyaev/ai_video/BackTestBench
configfile: pyproject.toml
plugins: locust-2.43.3, cov-7.1.0, Faker-38.2.0, anyio-3.7.1, asyncio-0.25.0
asyncio: mode=Mode.STRICT, asyncio_default_fixture_loop_scope=None
collected 206 items

tests/integration/test_get_candles.py . [ 0%]
tests/integration/test_ma_rsi_composable_e2e.py .. [ 1%]
tests/integration/test_market_data_cache.py . [ 1%]
tests/integration/test_strategy_engine_integration.py ... [ 3%]
tests/test_env_file.py ..... [ 6%]
tests/unit/analytics/test_analytics.py ..... [ 15%]
tests/unit/analytics/test_strategy_verdict.py .. [ 16%]
tests/unit/data_loader/test_backtest_fetch.py ..... [ 20%]
tests/unit/data_loader/test_loader.py ..... [ 23%]
tests/unit/data_loader/test_validator.py .... [ 25%]
tests/unit/db/test_session.py ... [ 27%]
tests/unit/engine/test_engine.py .... [ 29%]
tests/unit/engine/test_optimization_engine.py ..... [ 33%]
tests/unit/strategy/composable/test_compile.py ..... [ 42%]
tests/unit/strategy/composable/test_constraints.py ..... [ 45%]
tests/unit/strategy/composable/test_guard_and_trend.py ..... [ 48%]
tests/unit/strategy/composable/test_predicates_rules_actions.py ..... [ 51%]
tests/unit/strategy/composable/test_presets.py ..... [ 56%]
tests/unit/strategy/composable/test_series.py ..... [ 59%]
tests/unit/strategy/composable/test_time_predicates.py ..... [ 62%]
tests/unit/strategy/composable/test_trailing_stop.py .... [ 64%]
tests/unit/strategy/test_config_interface.py ..... [ 67%]
tests/unit/strategy/test_loader.py ... [ 69%]
tests/unit/strategy/test_ma_rsi.py ..... [ 71%]
tests/unit/strategy/test_rsi.py ..... [ 74%]
tests/unit/strategy/test_strategy.py ..... [ 83%]
tests/unit/test_backtest_config.py ..... [ 90%]
tests/unit/test_explore_robustness.py .... [ 92%]
tests/unit/test_live_run.py ... [ 93%]
tests/unit/test_strategy_manifest.py ..... [100%]

```

```

===== 206 passed in 1.91s =====

```

```

$ pytest tests --cov=src --cov-report=term -q
===== test session starts =====
platform darwin -- Python 3.12.2, pytest-8.3.4, pluggy-1.6.0
rootdir: /Users/georgijbelyaev/ai_video/BackTestBench
configfile: pyproject.toml
plugins: locust-2.43.3, cov-7.1.0, Faker-38.2.0, anyio-3.7.1, asyncio-0.25.0
asyncio: mode=Mode.STRICT, asyncio_default_fixture_loop_scope=None
collected 206 items

tests/integration/test_get_candles.py . [ 0%]
tests/integration/test_ma_rsi_composable_e2e.py .. [ 1%]
tests/integration/test_market_data_cache.py . [ 1%]
tests/integration/test_strategy_engine_integration.py ... [ 3%]
tests/test_env_file.py ..... [ 6%]
tests/unit/analytics/test_analytics.py ..... [ 15%]
tests/unit/analytics/test_strategy_verdict.py .. [ 16%]
tests/unit/data_loader/test_backtest_fetch.py ..... [ 20%]
tests/unit/data_loader/test_loader.py ..... [ 23%]
tests/unit/data_loader/test_validator.py .... [ 25%]
tests/unit/db/test_session.py ... [ 27%]
tests/unit/engine/test_engine.py .... [ 29%]
tests/unit/engine/test_optimization_engine.py ..... [ 33%]
tests/unit/strategy/composable/test_compile.py ..... [ 42%]
tests/unit/strategy/composable/test_constraints.py ..... [ 45%]
tests/unit/strategy/composable/test_guard_and_trend.py ..... [ 48%]
tests/unit/strategy/composable/test_predicates_rules_actions.py ..... [ 51%]
tests/unit/strategy/composable/test_presets.py ..... [ 56%]
tests/unit/strategy/composable/test_series.py ..... [ 59%]

```

```

tests/unit/strategy/composable/test_series.py ..... [ 59%]
tests/unit/strategy/composable/test_time_predicates.py ..... [ 62%]
tests/unit/strategy/composable/test_trailing_stop.py .... [ 64%]
tests/unit/strategy/test_config_interface.py ..... [ 67%]
tests/unit/strategy/test_loader.py ... [ 69%]
tests/unit/strategy/test_ma_rsi.py ..... [ 71%]
tests/unit/strategy/test_rsi.py ..... [ 74%]
tests/unit/strategy/test_strategy.py ..... [ 83%]
tests/unit/test_backtest_config.py ..... [ 90%]
tests/unit/test_explore_robustness.py .... [ 92%]
tests/unit/test_live_run.py ... [ 93%]
tests/unit/test_strategy_manifest.py ..... [100%]

```

```

===== tests coverage =====
_____ coverage: platform darwin, python 3.12.2-final-0 _____

```

Name	Stmts	Miss	Cover
src/__init__.py	0	0	100%
src/analytics/__init__.py	5	0	100%
src/analytics/metrics.py	194	42	78%
src/analytics/optimizer.py	82	19	77%
src/analytics/ranking.py	73	4	95%
src/analytics/strategy_verdict.py	37	2	95%
src/analytics/validation.py	51	0	100%
src/api/__init__.py	0	0	100%
src/api/app.py	0	0	100%
src/api/routers/__init__.py	0	0	100%
src/api/routers/backtest.py	0	0	100%
src/api/routers/data.py	0	0	100%
src/api/routers/strategies.py	0	0	100%
src/api/routers/top_n.py	0	0	100%
src/backtest_config.py	121	7	94%
src/broker_adapter/__init__.py	8	0	100%
src/broker_adapter/base.py	42	12	71%
src/broker_adapter/binance.py	82	60	27%
src/broker_adapter/bybit.py	87	67	23%
src/broker_adapter/csv_adapter.py	0	0	100%
src/broker_adapter/factory.py	43	19	56%
src/broker_adapter/models.py	23	0	100%
src/broker_adapter/tbank.py	149	114	23%
src/broker_adapter/twelvedata.py	87	68	22%
src/data_loader/__init__.py	6	0	100%
src/data_loader/backtest_fetch.py	107	7	93%
src/data_loader/cache.py	22	1	95%
src/data_loader/loader.py	133	17	87%
src/data_loader/models.py	8	1	88%
src/data_loader/normalizer.py	8	0	100%
src/data_loader/validator.py	31	1	97%
src/db/__init__.py	2	0	100%
src/db/models.py	14	0	100%
src/db/session.py	63	20	68%
src/engine/__init__.py	3	0	100%
src/engine/context.py	9	0	100%
src/engine/execution_config.py	10	1	90%
src/engine/execution_engine.py	32	0	100%
src/engine/models.py	76	0	100%
src/engine/optimization_engine.py	82	1	99%
src/engine/order_executor.py	48	3	94%
src/engine/portfolio.py	28	0	100%
src/engine/trading_bot.py	140	98	30%
src/engine/types.py	5	0	100%
src/env_file.py	61	6	90%
src/notifications/__init__.py	0	0	100%
src/notifications/email.py	0	0	100%
src/run_progress.py	29	9	69%
src/scheduler/__init__.py	0	0	100%
src/scheduler/scheduler.py	0	0	100%
src/strategy/__init__.py	12	0	100%
src/strategy/base.py	11	1	91%

src/strategy/composable/__init__.py	12	0	100%
src/strategy/composable/actions.py	75	4	95%
src/strategy/composable/compile.py	81	1	99%
src/strategy/composable/constraints.py	37	4	89%
src/strategy/composable/context.py	61	11	82%
src/strategy/composable/definition.py	66	7	89%
src/strategy/composable/errors.py	3	0	100%
src/strategy/composable/predicates.py	164	18	89%
src/strategy/composable/presets.py	26	0	100%
src/strategy/composable/rules.py	51	5	90%
src/strategy/composable/series.py	113	6	95%
src/strategy/composable/strategy.py	82	8	90%
src/strategy/config.py	46	6	87%
src/strategy/errors.py	5	0	100%
src/strategy/loader.py	37	9	76%
src/strategy/registry.py	36	4	89%
src/strategy/schema.py	56	6	89%
src/strategy/store.py	45	6	87%
src/strategy/strategies/__init__.py	3	0	100%
src/strategy/strategies/ma_crossover.py	50	7	86%
src/strategy/strategies/ma_rsi.py	80	11	86%
src/strategy/strategies/rsi_threshold.py	69	11	84%
src/strategy_manifest.py	91	2	98%
src/token_validation.py	43	33	23%
=====			
TOTAL	3356	739	78%
===== 206 passed in 5.33s =====			

Pytest summary and src coverage – 206 passed, 78% (submission freeze 0902752)

```

name: PR Build & Run

on:
  pull_request:
    branches: [main]

jobs:
  backend-tests:
    runs-on: ubuntu-latest
    if: github.event.action != 'closed'

    steps:
      - name: Checkout PR
        uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.12"
          cache: pip
          cache-dependency-path: requirements.txt

      - name: Install Python dependencies
        run: |
          python -m pip install --upgrade pip
          python -m pip install -r requirements.txt

      - name: Run backend tests
        run: pytest tests -q

  frontend-checks:
    runs-on: ubuntu-latest
    if: github.event.action != 'closed'

    steps:
      - name: Checkout PR
        uses: actions/checkout@v4

      - name: Set up Node.js
        uses: actions/setup-node@v4
        with:
          node-version: "20"
          cache: npm
          cache-dependency-path: frontend/package-lock.json

      - name: Install and build frontend
        working-directory: frontend
        run: |
          npm ci
          npm run build

      - name: Lint frontend (non-blocking until page.tsx refactor)
        working-directory: frontend
        continue-on-error: true
        run: npm run lint

  docker-smoke:
    runs-on: ubuntu-latest
    if: github.event.action != 'closed'

    env:
      APP_PORT: "8080"

    steps:
      - name: Checkout PR
        uses: actions/checkout@v4

      - name: Prepare .env for Compose
        run: |
          cat > .env <<EOF
          TINKOFF_TOKEN=${{ secrets.TINKOFF_TOKEN }}
          TWELVEDATA_TOKEN=${{ secrets.TWELVEDATA_TOKEN }}
          EOF

      - name: Build and start via docker compose
        run: docker compose up -d --build

      - name: Smoke-check (containers up + HTTP 200)
        run: |
          sleep 10
          docker compose ps
          EXITED=$(docker compose ps --status exited -q)
          [ -z "$EXITED" ]
          curl -fsS "http://localhost:${APP_PORT}/"

      - name: Tear down stack
        if: always()
        run: docker compose down --remove-orphans 2>/dev/null || true

```

10.2 Verification (re-measured 20.07.2026 on 0902752)

```
pytest tests -q

===== 206 passed in ~2.8s =====

pytest tests --cov=src --cov-report=term

TOTAL ... 78% (206 passed)
```

Frontend and Docker smoke remain covered by the GitHub Actions matrix on [PRs](#) (backend-tests, frontend-checks, docker-smoke).

10.3 Selected [PR](#) index (product evolution)

Window	PRs (examples)
Week 4	#98 strategies, #100 ranking/validation lib, #101–#104 dashboard/data
Week 5	#127 composable, #128/#130 optimizer+UX, #135–#137 brokers/CI
Week 6	#139 ranked[], #141 tokens, #142 constraints, #143/#146 explore/frontend, #144 bot, #145 Binance
Post-W6	#150 OpenAPI + Swagger UI , #151 bootstrap progress + Live refresh + strategy health + dock removal

10.4 Full individual contributions (PR audit trail)

Complete per-member [PR](#) lists (same content as §1.1, kept here for audit without scrolling the main narrative):

Member	Role focus	Full deliverables & evidence
Marat Diarov	Team lead · frontend ·	PM: Kanban board , Issues , weekly customer demos (transcriptions/ ; demo transcript #134); weekly PDF reports W2–W6 (§8); this Final Report & Final presentation.pdf . Frontend: #90 (dashboard MVP–1 + Docker integration), #101–#102 (multi-strategy dashboard & ranking UI), #130 (strategy integration, grid search, run/stop routes), #141 (multi-API tokens panel), #143 (Explore workflow dock + stability analytic

Member	Role focus	Full deliverables & evidence
	repo	s), #146 (Bot workflow dock – explore/bot docks removed on submission UI), #151 (bootstrap progress, chunked candle fetch, Live refresh, strategy health UI). DevOps: Docker Node.js deployment smoke #96 . Data path follow-up: normalization & validation pipeline #104 . Design: Figma prototype ; UI screenshots §10.1. Docs: backtesting reference PDFs #81 ; composable engine design spike #107 .
Georgi Belyaev	Tech lead · architecture · DevOps/QA	Architecture & requirements: interfaces hub & project structure #75 ; living docs hub audits #95 , #105 , #138 . DevOps/CI: self-hosted Docker smoke #91 , port/profile fixes #92–#93 ; GitHub-hosted three-job matrix #137 . Docs/API: OpenAPI spec + Swagger UI #150 (live docs). QA: pytest/coverage baselines (§7.2, §10.2); co-authored integrated Docker path in #90 ; Final Report compilation & freeze verification.
Bogdan Zinchenko	Data engineering · historical data	Data Loader foundation: initial loader, validation, SQLite candle model, cache module – #97 (src/data_loader/). Evidence: src/data_loader/validator.py, cache.py, SQLite upsert used by bootstrap and optimizer (§3.1); chunked fetch path in backtest_fetch.py integrated at submission freeze (#151 , led by Marat).
Samy Badaoui	Simulation engine	Core engine: candle-by-candle long-only simulation, portfolio model, trade log – #82 (src/engine/). Optimizer execution: random-search engine and optimization module tests – #128 . Validation library: minimal out-of-sample trading-bot pipeline – #144 (src/engine/trading_bot.py; library + tests; UI dock removed on submission HEAD). Evidence: integration tests under tests/integration/; incremental engine path consumed by Live refresh in #151 .
Maxim Kharlamov	Analytics · ranking	Metrics spec & MVP-1 analytics: core performance metrics #77 , MVP-1 pipeline #89 . Ranking & validation library: Top-N ranking and validation metrics module #100 . Optimizer ranking: optimization.ranked[] schema and ranking helpers #139 . Submission freeze: strategy health verdict surfaced on dashboard cards (#151 , led by Marat). Docs: docs/analytics data model specification.md , docs/core performance metrics.md .
Philipp Oghenevwe I	Strategy module	Strategy architecture & MVP-1: registry, config parser, MA Crossover – #83 ; architecture note #76 . Plugin extensibility: auto-discovery, MA+RSI, RSI threshold, parameter specs – #98 . Composable YAML engine: series/rules/actions compile path – #127 . Risk controls: constraints, trailing stop, time/drawdown/trend filters – #142 . Docs: docs/strategy module plugins and configuration.md .

Member	Role focus	Full deliverables & evidence
diare		
Nikolay Taran	Broker adapter · integrations	T-Bank adapter: first live historical-candle integration – #88 . Multi-broker expansion: TwelveData / Bybit examples and timestamp contract – #135 ; T-Bank usage docs + candle parameter validation – #103 . Binance adapter: candle adapter + usage examples – #145 . Schema/docs: broker adapter and database_schema.md (#80). Evidence: multi-API tokens panel §10.1; factory wiring in src/broker_adapter/factory.py.

11. Submission verification

Pre-submission checklist performed by **Georgii Beliaev** (tech lead) and **Marat Diarov** (team lead) on **20.07.2026** against freeze commit 09027526330bc9b94208f302f669623168826bef (0902752):

- ☒ Full PDF read end-to-end (this document → Final report.pdf)
- ☒ Hyperlinks exercised: GitHub repo/commit, VM [/](#) and [/docs](#), Kanban, Issues, Figma prototype, weekly PDFs (§8), OpenAPI on VM
- ☒ Local checkout at 0902752: pytest tests -q → **206 passed**; --cov=src → **78%** (§10.2)
- ☒ Screenshots in §4 and §10 captured from docker compose up --build at 0902752
- ☒ Final demo video recorded at 0902752 UI; uploaded to **course portal** (primary)

Conclusion

BackTestBench delivered an industrial research sandbox: multi-broker historical data, composable strategies with optimization and ranking, strategy health verdicts, optional Live refresh, and a Dockerized dashboard for bootstrap backtests on one screen. Weekly customer feedback steered the product from a single hardcoded path to a modular bench; post-Week 6 work simplified the UI (Explore/Bot docks removed) while adding bootstrap progress, health checks, and OpenAPI docs. Remaining gaps (preset wiring reliability, full walk-forward / multi-period validation, percentage/risk-reward UX polish) are documented as the honest next steps – not as completed claims.